

Kmos-10020

SUMMARY

Version: 1.0 Status: Constitutional Engineering Document Audience: Principal Engineers; AI Coding Agents; Platform Maintainers; Technical Architects

PURPOSE

This document defines the constitutional engineering rules governing the KMOS source repositories. The repository is not merely a storage location for source code. It is the physical representation of the platform architecture. A well-organized repository reinforces architectural integrity. A poorly organized repository encourages architectural erosion. Every implementation SHALL preserve this constitution.

REPOSITORY PHILOSOPHY

The repository SHALL reflect business architecture rather than implementation technology. Developers should understand the KMOS architecture simply by exploring the repository. Folders SHALL represent architectural concepts. Not programming languages. Not frameworks. Not deployment technologies.

REPOSITORY GOALS

The repository SHALL: Teach the architecture. Encourage modular development. Prevent architectural drift. Support independent teams. Support autonomous AI development. Support long-term maintainability. Remain understandable for decades.

CONSTITUTIONAL PRINCIPLES

The repository SHALL organize work by: Business Responsibility; Institutional Engine; Capability; Domain; Application; Extensions; Infrastructure; Documentation. NOT by: Framework; Programming Language; Vendor; Cloud Provider; Database. Implementation technologies SHALL remain secondary.

TOP-LEVEL REPOSITORY LAYOUT

Reference organization. kmos/ ├── specifications/ ├── constitution/ ├── architecture/ ├── platform/ ├── engines/ ├── capabilities/ ├── domains/ ├── applications/ ├── extensions/ ├── connectors/ ├── sdk/ ├── tools/ ├── deployment/ ├── governance/ ├── testing/ ├── examples/ ├── documentation/ └── scripts/ The logical organization SHALL remain stable even if physical repositories evolve.

SPECIFICATIONS

The specifications directory SHALL contain only normative architectural documents. Examples: KMOS-0001, KMOS-0100, KMOS-0201. Implementation SHALL never modify specifications directly without governance review.

CONSTITUTION

The Constitution directory SHALL contain: Implementation Constitution; Repository Constitution; Engineering Charter; Coding Constitution; Governance Constitution. These documents define engineering behavior.

ARCHITECTURE

Architecture contains: Reference Diagrams; Data Models; Sequence Diagrams; Deployment Models; Object Catalogs; Event Catalogs; Reference Architectures. Architecture SHALL remain implementation independent.

PLATFORM

The platform directory contains foundational platform services. Examples: Knowledge; Assets; Events; Identity; Governance; Workflow; Capability Registry; Configuration; Search. Each SHALL remain independently deployable.

ENGINES

The engines directory contains reusable execution engines. Examples: Workflow Runtime; Capability Runtime; Policy Evaluation; Semantic Engine; Rendering Engine; Scheduling Engine; Notification Engine; Search Engine. An engine provides reusable execution infrastructure.

CAPABILITIES

Capabilities SHALL contain reusable business logic. Examples: Speech Recognition; Translation; Vocabulary Expansion; Knowledge Extraction; Rendering; Publishing; Archiving; Summarization. Business logic SHALL reside here.

DOMAINS

Domains compose Capabilities into business solutions. Examples: Media; Education; Research; Publishing; Preservation; Learning; Healthcare; Government. Domains SHALL remain independent.

APPLICATIONS

Applications provide user experiences. Examples: Studio; Research Portal; Administration; Archive Explorer; Mobile. Applications SHALL remain thin. Business logic SHALL remain outside applications.

EXTENSIONS

Extensions package reusable platform additions. Examples: Publishing Extensions; Language Packs; Connector Packages; Knowledge Packs; Reference Implementations. Extensions SHALL remain independently installable.

CONNECTORS

Connectors integrate external systems. Examples: Cloud Storage; Learning Platforms; Publishing Systems; Identity Providers; AI Providers; ERP Systems. Connectors SHALL translate external protocols into canonical KMOS contracts.

SDK

SDK SHALL contain: Client Libraries; Capability Templates; Extension Templates; Testing Utilities; Developer Tools. The SDK SHALL support implementation rather than architecture.

TESTING

Testing SHALL mirror architecture. Examples: Platform Tests; Capability Tests; Workflow Tests; Integration Tests; Replay Tests; Governance Tests; Performance Tests. Testing SHALL remain architecture-aware.

DOCUMENTATION

Documentation SHALL include: Architecture; Developer Guides; Operations; Deployment; Tutorials; Examples; Migration. Documentation SHALL evolve continuously.

DEPLOYMENT

Deployment SHALL contain: Containers; Kubernetes; Helm; Infrastructure; CI/CD; Monitoring. Deployment SHALL remain replaceable.

CODING STANDARDS

Every implementation SHALL: Separate domain from infrastructure. Separate APIs from business logic. Separate contracts from implementations. Avoid circular dependencies. Avoid shared mutable state. Prefer explicit interfaces. Coding SHALL reinforce architecture.

DEPENDENCY RULES

Allowed dependency direction: Applications ↓ Domains ↓ Capabilities ↓ Platform Services ↓ Infrastructure. Reverse dependencies SHALL require architectural justification.

AI CODING RULES

Autonomous coding agents SHALL: Create one module at a time. Maintain architectural boundaries. Never duplicate business logic. Generate tests. Generate documentation. Keep commits focused. Document architectural decisions. The repository SHALL remain understandable after every commit.

DEFINITION OF DONE

A repository contribution is complete only when: Code implemented. Tests passing. Documentation updated. Events documented. Objects documented. Architecture validated. Governance approved. Deployment verified. Repository integrity preserved.

REPOSITORY GOVERNANCE

Major structural changes REQUIRE: Architecture Review. Migration Plan. Backward Compatibility Analysis. Documentation Update. Governance Approval. The repository architecture SHALL evolve deliberately.

REALITY CHECK

This repository structure reflects lessons learned from previous implementations. Validated concepts include: Capability-first engineering; Workflow orchestration; Asset lineage; Governance; Institutional knowledge. Areas expected to evolve include: Marketplace; Federation; Advanced AI orchestration; Cross-organization collaboration. The repository SHALL evolve from proven engineering rather than speculative organization.

LONG-TERM OBJECTIVE

The repository should remain understandable by engineers who were not present during its creation. Its organization should naturally communicate: What the platform does. How it is organized. Where responsibilities belong. How new functionality should be added. The repository itself becomes an architectural teaching tool.

RELATIONSHIP TO PREVIOUS DOCUMENTS

This document complements: KMOS-9999 Implementation Constitution; KMOS-10000 Implementation Dossier; KMOS-10005 Product Vision & Engineering Charter; KMOS-10010 Master Implementation Roadmap. Where those documents define implementation philosophy, this Repository Constitution defines the physical organization through which that philosophy is expressed.

CLOSING STATEMENT

The KMOS repository is more than source code. It is the long-term institutional memory of the engineering effort itself. Future contributors—human or AI—should be able to understand the platform architecture, navigate its components, and extend it confidently without relying on tribal knowledge. Every directory, module, package, and file should reinforce the principles of clarity, composability, governance, and institutional preservation that define KMOS.